Microprocessor and Assembly Language

CSE 3321

Organization of the 8086/8088 Microprocessors

Presented By

Saifur Rahman

Lecturer

Dept. of CSE

NBIU

References

Assembly Language Programming and Organization of the IBM PC

By Ytha Yu and Charles Maru

The 8086 and 8088 Microprocessors

- Intel introduced the 8086 in 1978 as its **first 16-bit microprocessor**.
- The 8088 was introduced in 1979 and is essentially the same as the 8086 internally.
- Externally, the 8086 has a **16-bit data bus**, while the 8088 has an **8-bit data bus**.
- The 8086 operates at a **faster clock rate**, resulting in **better performance** compared to the 8088.
- IBM chose the 8088 for the original PC because it was **less expensive** to build a computer around.
- Both the 8086 and 8088 share the **same instruction set**, forming the basis for other microprocessors in the family.

Registers

- Registers inside the microprocessor **store information**.
- Data registers **hold data** for operations.
- Address registers **store addresses** of instructions or data.
- The instruction register keeps **track of the current state** of the processor.
- The 8086 has **4 general data registers** and **3 types of address registers**.
- There's also a **status register** named **FLAGS**.
- Altogether, there are **fourteen 16-bit registers**.
- Specific functions of these registers will become clearer with practice.

Data Registers: AX, BX, CX, DX

- These four registers are for **general data manipulation**.
- Storing data in registers **speeds up operations** compared to memory.
- Modern processors have many registers for efficiency.
- Data registers' high and low bytes can be accessed separately.
- For example, **AX** has **AH (high)** and **AL (low)**, **BX** has **BH** and **BL**, **CX** has **CH** and **CL**, and **DX** has **DH** and **DL**.
- This setup provides more registers for handling byte-size data.

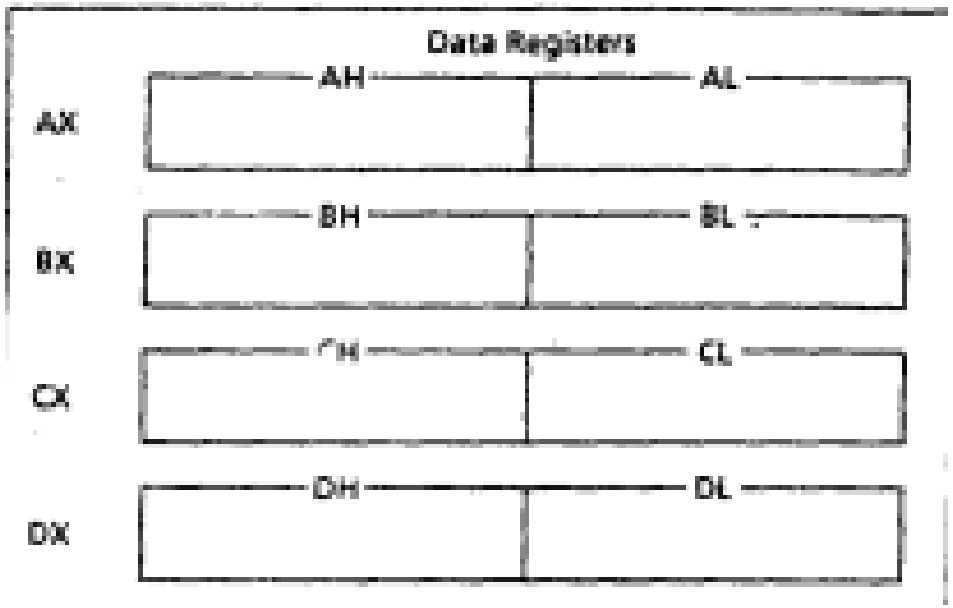

Fig 1: Data Registers

Data Registers: AX, BX, CX, DX (cont'd)

- **AX:** Preferred for **arithmetic, logic, and data transfer**; generates efficient machine code.
- **BX:** Functions as an **address register**; used in operations like **table look-up** (e.g., XLAT).
- **CX:** Utilized as a **loop counter** in program loop constructions and string operations; **CL** counts in **bit shift/rotate** instructions.
- **DX:** Used in **multiplication**, **division**, and **I/O operations**.

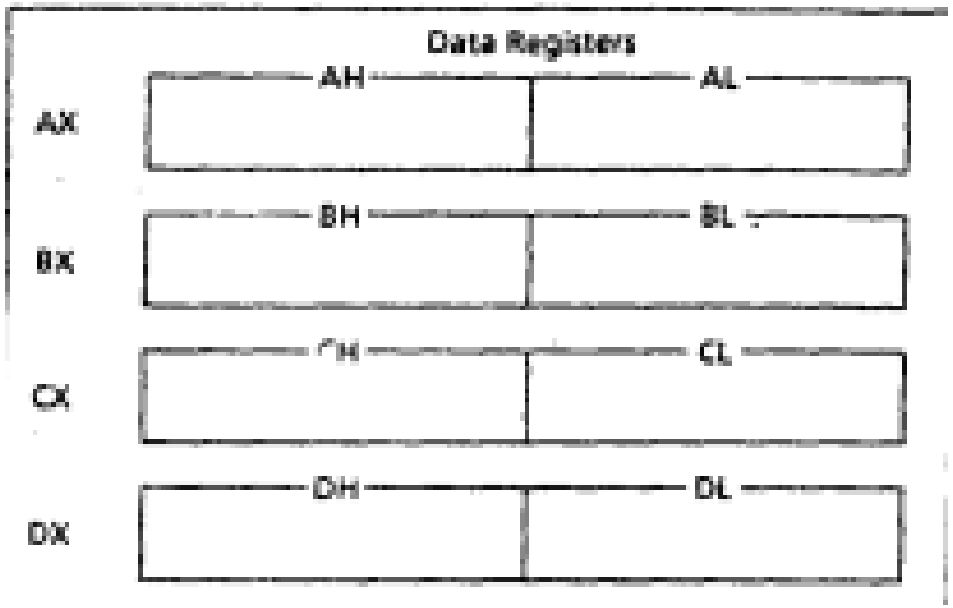

Fig 1: Data Registers

Segment Registers: CS, DS, SS, ES

- Address registers store memory addresses used by the processor to access data and instructions.
- Memory in the 8086 is organized into bytes, each with a unique address starting from 0.
- The 8086 processor utilizes a **20-bit physical address**, allowing access to **1,048,576 bytes (one megabyte)** of memory.
- Addresses are often expressed in hexadecimal format for simplicity.
- For example, the first five memory addresses are: 00000h, 00001h, 00002h, 00003h, 00004h, and so on up to FFFFFh.
- The **addresses are too big (20-bit)** to fit in a 16-bit register or memory word.
- Due to the limitations of a 16-bit processor, **memory segmentation** is employed, **dividing memory into segments to accommodate the 20-bit addresses**.

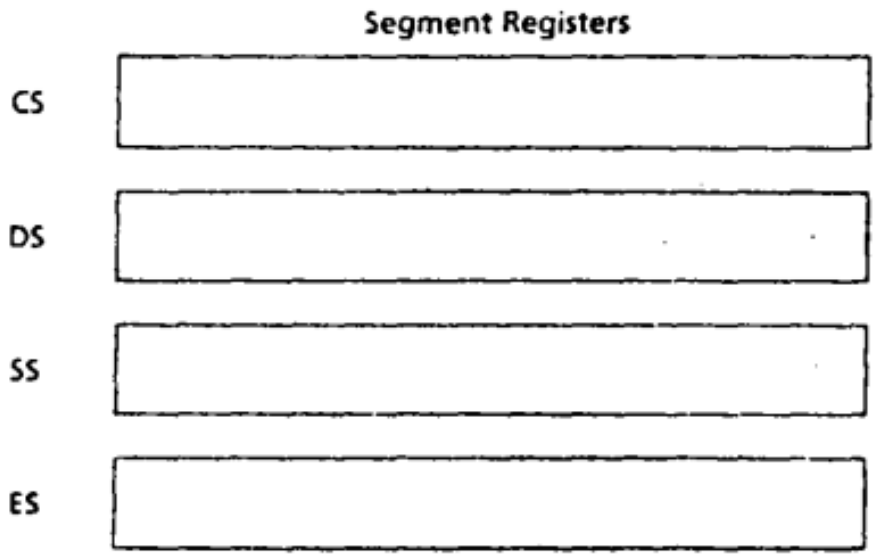

Fig 2: Segment Registers

Memory Segments

- **Memory segment:** A block of 64 KB memory bytes, identified by a segment number.
- **Segment number: 16-bit identifier**, ranging from 0 to FFFFh.
- Within a segment: Memory location specified by an **offset**, a **16-bit number**.
- Logical address: **Segment : Offset** format.
- Example: **A4FB : 4872h** specifies **offset 4872h** in **segment A4FBh**.
- **Physical address:** Obtained by **shifting the segment address 4 bits left (multiplying by 10h)** and **adding the offset.**
- Example: A4FB0h + 4872h = A9822h (20-bit physical address).

```
 A4FB0h
+ 4872h
 ------
 A9822h   (20–bit physical address)
```

Program Segments

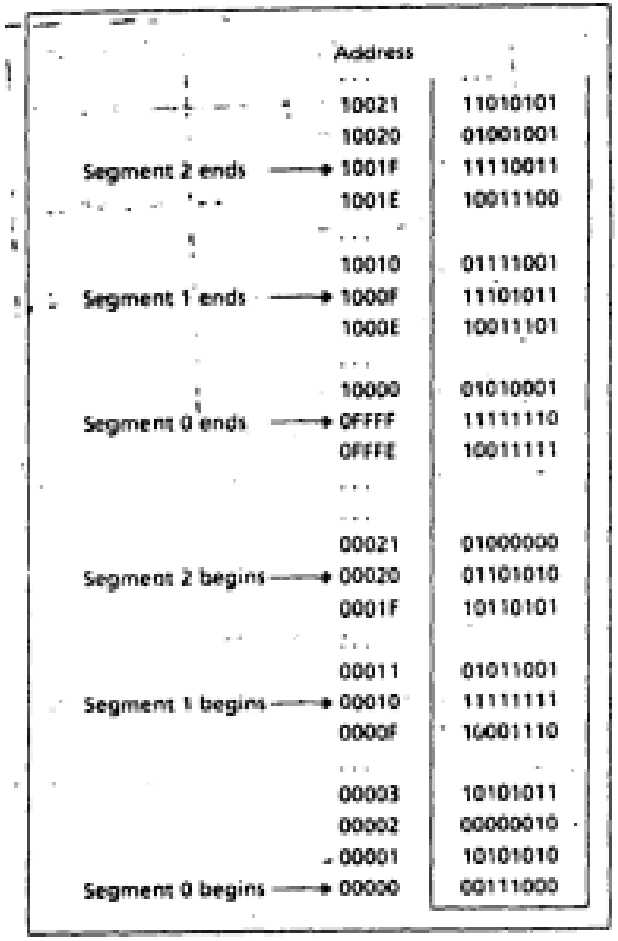

Fig 3: Location of Segments

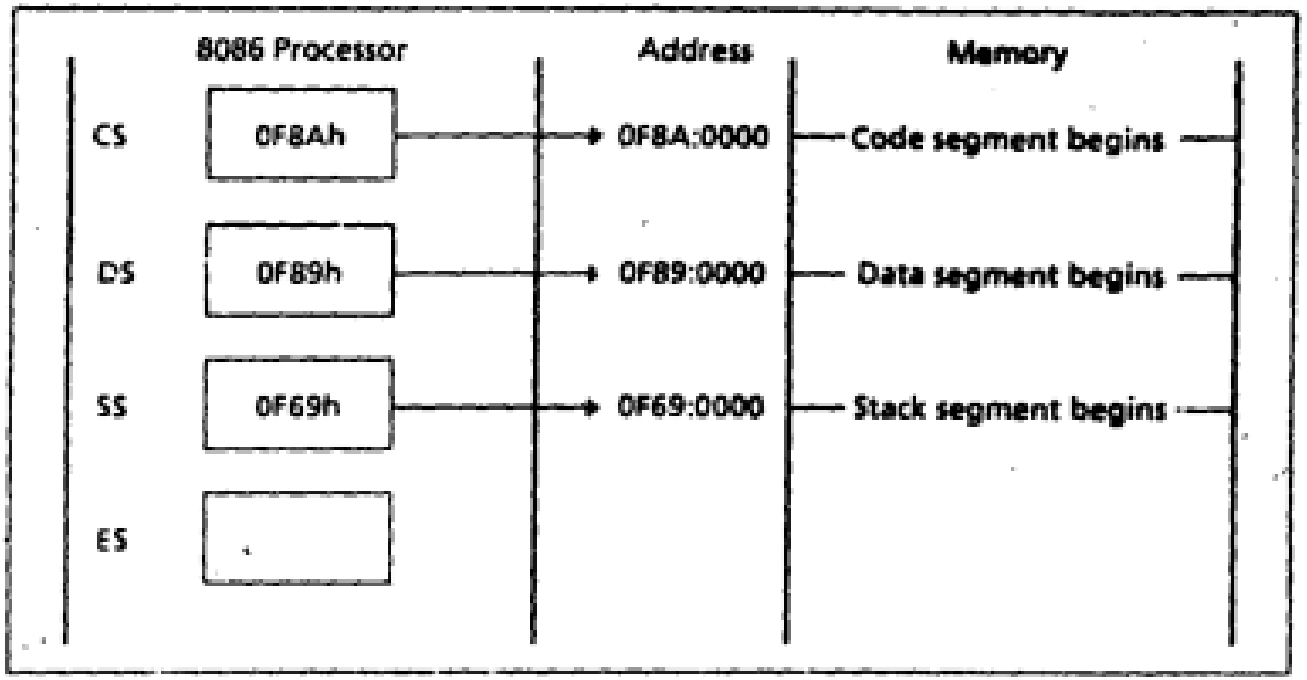

Fig 4: Segment Registers

Program Segments (cont'd)

- **CS, DS, SS,** and **ES** are segment registers in the 8086 processor.
- They hold **segment numbers** for **code, data, stack**, and **extra segments** respectively.
- Code, data, and stack segments are distinct areas for **program execution**, **data storage**, and **stack implementation**.
- The **ES register** is used for accessing **additional data segments** if needed.
- Program segments don't have to occupy **the entire 64 KB** in a memory segment.
- Memory segments can **overlap**, allowing **smaller program segments** to be **placed close together**.
- At any time, **only memory locations addressed by the four segment registers are accessible**, making only four memory segments active.
- However, **programs can modify segment register contents** to address different segments as needed.

Pointer and Index Registers: SP, BP, SI, DI

- **SP, BP, SI,** and **DI** registers typically contain **offset addresses** of memory locations.
- **Unlike segment registers**, pointer and index registers (SP, BP, SI, and DI) can be used in **arithmetic and other operations**.

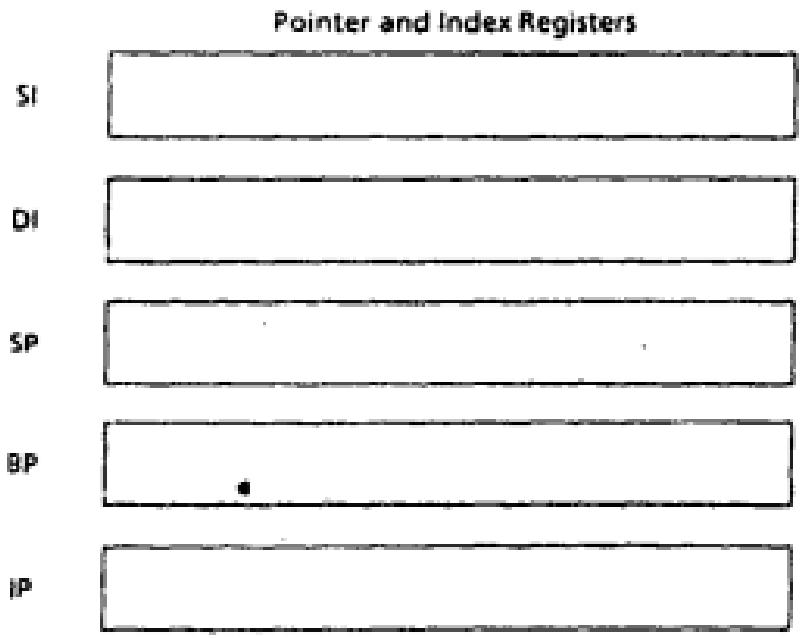

Fig 5: Pointer and Index Registers

Pointer & Index Registers: SP, BP, SI, DI (cont'd)

- **SP (Stack Pointer):** Used with **SS** to access the **stack segment**.
- **BP (Base Pointer):** Primarily used for **accessing data on the stack**; can also access data in other segments.
- **SI (Source Index):** Points to memory locations in the data segment addressed by **DS**; incremented to access consecutive memory locations.
- **DI (Destination Index):** Performs similar functions as SI; used in string operations to access memory locations addressed by **ES**.

Instruction Pointer

- To access instructions, the 8086 uses **CS and IP** registers.
- **CS (Code Segment)** contains the **segment number** of the next instruction.
- **IP (Instruction Pointer)** contains **the offset** and is **updated after each instruction** execution to point to the next instruction.
- Unlike other registers, IP cannot be directly manipulated by an instruction; it cannot be used as an operand.

Flag Registers

- The FLAGS register indicates **the status of the microprocessor** through **individual bits** called flags.
- There are two types of flags: **status flags** and **control flags.**
- **Status flags** reflect **the result of instructions** executed by the processor. For example, the **ZF (zero flag)** is set to **1** when a subtraction operation **results in 0**.
- Control flags enable or disable specific operations of the processor. For instance, if the **IF (interrupt flag)** is **cleared (set to 0)**, inputs from the keyboard are **ignored** by the processor.

The Operating System

- The operating system (OS) manages computer devices and functions.
- Functions include user command execution, I/O operations, error handling, and resource management.
- **Disk Operating System (DOS)** is popular for IBM PC, **designed for 8086/8088-based computers**.
- DOS **handles up to 1 MB of memory**, **lacks multitasking**, but runs on later processors in real address mode.
- DOS organizes disk information into named files with extensions.
- Versions 2.1 or later are often needed for commercial programs.
- DOS consists of service routines accessed through user commands, supporting a GUI in DOS 5.0.
- **COMMAND.COM** handles **user commands, generates prompts, and executes them**.
- Commands can be internal (in memory) or external (referring to unloaded routines or programs).
- **BIOS routines** in **ROM** provide basic system functions even when the computer is off.

BIOS: Basic Input/Output System

- First Program to run when the computer starts.
- BIOS routines **handle I/O operations** for the PC and are machine-specific, varying by PC model.
- Each PC model has its own BIOS routines, which interact with the machine's I/O port registers.
- DOS I/O operations are ultimately executed by BIOS routines.
- BIOS also performs **circuit checking** and **loads DOS routines**.
- Interrupt vectors, containing BIOS routine addresses, start at memory address 00000h to allow DOS and other programs to utilize BIOS functions.
- IBM copyrights its BIOS routines, so IBM-compatible PCs use their own BIOS routines, with compatibility depending on how closely they match IBM's BIOS.

The Start-up Operation

- When the PC is powered up, the 8086/8088 processor **enters a reset state**.
- **CS register** is set to **FFFFh**, and **IP** is set to **0000h**, so the first instruction executes at memory location **FFFF0h**, which is in **ROM**.
- **FFFF0h** contains an instruction to **transfer control** to the starting point of the **BIOS routines**.
- BIOS routines initially **check for system and memory errors**, then initialize **interrupt vectors and BIOS data area**.
- BIOS proceeds to **load the operating system** from the system disk in **two steps**:
  - Loading a small program called the **boot program**.
  - Boot program **loads the actual operating system** routines.
- The boot program is named for being part of the OS; it isolates BIOS from OS changes and keeps BIOS size smaller.
- After loading the OS into memory, **COMMAND.COM** gains control.